

Лістинг коду програми

Вихідні файли також доступні за посиланням <https://github.com/Teollan/2026-comparch-lab3/tree/lab4>.

main.cpp

```
1 #include <cstdint>
2 #include <fstream>
3 #include <iostream>
4 #include <iterator>
5 #include <optional>
6 #include <sstream>
7 #include <string>
8 #include <utility>
9 #include <vector>
10
11 #include "assembly/Decompiler/Decompiler.hpp"
12 #include "assembly/Instruction/Instruction.hpp"
13 #include "assembly/InstructionFormat/InstructionFormat.hpp"
14 #include "util/Args/Args.hpp"
15 #include "util/Hex/Hex.hpp"
16 #include "vram/VramImage/VramImage.hpp"
17
18 std::optional<Args> tryParseArgs(int argc, char* argv[]) {
19     Args args(argc, argv);
20     args.expectString("input", "i", true)
21         .expectString("segments-table", "s", true)
22         .expectFlag("hex", "x")
23         .expectString("output", "o");
24
25     try {
26         args.parse();
27     } catch (const ArgsException& e) {
28         std::cerr << e.what() << "\n";
29         args.printUsage();
30         return std::nullopt;
31     }
32
33     return args;
34 }
35
36 std::optional<std::vector<std::uint8_t>> tryReadInput(const Args& args) {
37     std::ifstream input(args.getString("input"), std::ios::binary);
38     if (!input) {
39         std::cerr << "Failed to open file: " << args.getString("input") << "\n";
40         return std::nullopt;
41     }
42
43     if (args.getFlag("hex")) {
44         std::stringstream buffer;
45         buffer << input.rdbuf();
46         return hexToBytes(buffer.str());
47     } else {
48         std::vector<std::uint8_t> bytes;
49         bytes.assign(
50             std::istreambuf_iterator<char>(input),
```

```

51         std::istreambuf_iterator<char>()
52     );
53     return bytes;
54 }
55 }
56
57 std::optional<Vram> tryLoadVramImage(const Args& args) {
58     try {
59         return VramImage::load(args.getString("segments-table"));
60     } catch (const std::exception& e) {
61         std::cerr << "Failed to load VRAM image: " << e.what() << "\n";
62         return std::nullopt;
63     }
64 }
65
66 std::optional<std::vector<Instruction>> tryDisassembling(
67     std::vector<std::uint8_t>& bytes,
68     const Vram& vram
69 ) {
70     try {
71         return decompile(bytes, vram);
72     } catch (const std::exception& e) {
73         std::cerr << "Failed to disassemble bytes: " << e.what() << "\n";
74         return std::nullopt;
75     }
76 }
77
78 int main(int argc, char* argv[]) {
79     auto args = tryParseArgs(argc, argv);
80     if (!args) {
81         return 1;
82     }
83
84     auto bytes = tryReadInput(*args);
85     if (!bytes) {
86         return 1;
87     }
88
89     auto vram = tryLoadVramImage(*args);
90     if (!vram) {
91         return 1;
92     }
93
94     auto instructions = tryDisassembling(*bytes, *vram);
95     if (!instructions) {
96         return 1;
97     }
98
99     std::ofstream outputFile;
100     std::ostream* out = &std::cout;
101
102     const auto& outputPath = args->getString("output");
103     if (!outputPath.empty()) {
104         outputFile.open(outputPath);
105
106         if (!outputFile) {
107             std::cerr << "Failed to open output: " << outputPath << "\n";
108             return 1;
109         }
110

```

```

111     out = &outputFile;
112 }
113
114 for (const auto& instruction : *instructions) {
115     *out << bytesToHex(*bytes, instruction.offset(), instruction.size())
116         << "\n";
117
118     if (auto failure = instruction.error()) {
119         *out << "ERROR: "
120             << describeTranslationError(
121                 failure->virtualAddress, failure->error
122             )
123             << "\n";
124     }
125
126     *out << formatInstruction(instruction) << "\n\n";
127 }
128
129 return 0;
130 }

```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <optional>
5 #include <utility>
6 #include <vector>
7
8 #include "assembly/Instruction/Instruction.hpp"
9 #include "vram/Vram/Vram.hpp"
10
11 std::vector<Instruction> decompile(
12     std::vector<std::uint8_t>& bytes,
13     const Vram& vram
14 );
15
16 class DecompilerUnknownInstructionException : public std::exception {
17 private:
18     std::size_t offset;
19
20 public:
21     DecompilerUnknownInstructionException(std::size_t offset);
22 };
```

```
1 #include "assembly/Decompiler/Decompiler.hpp"
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <optional>
6 #include <utility>
7 #include <vector>
8
9 std::vector<Instruction> decompile(
10     std::vector<std::uint8_t>& bytes,
11     const Vram& vram
12 ) {
13     std::vector<Instruction> instructions;
14
15     auto iterator = bytes.begin();
16     while (iterator != bytes.end()) {
17         std::size_t offset = iterator - bytes.begin();
18         std::optional<Instruction> instruction = std::nullopt;
19
20         for (const auto& prototype : INSTRUCTION_PROTOTYPES) {
21             auto instructionBytes = std::vector<std::uint8_t>(
22                 iterator, iterator + prototype.size()
23             );
24
25             if (prototype.doesMatch(instructionBytes)) {
26                 instruction.emplace(
27                     prototype, std::move(instructionBytes), offset, vram
28                 );
29
30                 break;
31             }
32         }
33
34         if (!instruction) {
35             throw DecompilerUnknownInstructionException(offset);
36         }
37
38         iterator += instruction->size();
39         instructions.push_back(std::move(*instruction));
40     };
41
42     return instructions;
43 }
44
45 DecompilerUnknownInstructionException::DecompilerUnknownInstructionException(
46     std::size_t offset
47 ) :
48     offset(offset) {}
```

```
1 #include "assembly/Decompiler/Decompiler.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <memory>
6 #include <vector>
7
8 #include "vram/PagesTable/PagesTable.hpp"
9 #include "vram/SegmentsTable/SegmentsTable.hpp"
10
11 using Bytes = std::vector<std::uint8_t>;
12
13 namespace {
14
15 Vram makeIdentityVram() {
16     std::vector<PageDescriptor> pages;
17     pages.push_back(PageDescriptor{.isResident = true, .frame = 0});
18
19     std::vector<SegmentDescriptor> segments;
20     segments.push_back(
21         SegmentDescriptor{
22             .pagesTable = std::make_unique<PagesTable>(std::move(pages))
23         }
24     );
25
26     return Vram(std::make_unique<SegmentsTable>(std::move(segments)));
27 }
28
29 } // namespace
30
31 TEST(decompile, EmptyInputReturnsEmptyResult) {
32     Vram vram = makeIdentityVram();
33     Bytes bytes;
34     auto result = decompile(bytes, vram);
35
36     EXPECT_TRUE(result.empty());
37 }
38
39 TEST(decompile, SingleTwoByteInstruction) {
40     Vram vram = makeIdentityVram();
41     Bytes bytes{0x0A, 0x12};
42     auto result = decompile(bytes, vram);
43
44     ASSERT_EQ(result.size(), 1);
45     EXPECT_EQ(result[0].prototype.opcode, 0x0A);
46     EXPECT_EQ(result[0].offset(), 0u);
47     EXPECT_EQ(result[0].size(), 2u);
48 }
49
50 TEST(decompile, SingleThreeByteInstruction) {
51     Vram vram = makeIdentityVram();
52     Bytes bytes{0x03, 0x01, 0x23};
53     auto result = decompile(bytes, vram);
54
55     ASSERT_EQ(result.size(), 1);
56     EXPECT_EQ(result[0].prototype.opcode, 0x030);
```

```

57 }
58
59 TEST(decompile, SingleSixByteInstruction) {
60     Vram vram = makeIdentityVram();
61     Bytes bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
62     auto result = decompile(bytes, vram);
63
64     ASSERT_EQ(result.size(), 1);
65     EXPECT_EQ(result[0].prototype.opcode, 0x1B0);
66 }
67
68 TEST(decompile, MultipleInstructionsSameSize) {
69     Vram vram = makeIdentityVram();
70     Bytes bytes{0x0A, 0x12, 0x80, 0xAF};
71     auto result = decompile(bytes, vram);
72
73     ASSERT_EQ(result.size(), 2);
74     EXPECT_EQ(result[0].prototype.opcode, 0x0A);
75     EXPECT_EQ(result[1].prototype.opcode, 0x80);
76 }
77
78 TEST(decompile, OffsetsTrackInstructionStarts) {
79     Vram vram = makeIdentityVram();
80     Bytes bytes{
81         0x0A,
82         0x12, // SUB R1, R2
83         0x03,
84         0x01,
85         0x23, // ADD R1, R2, R3
86         0x1B,
87         0x01,
88         0x00,
89         0x00,
90         0x00,
91         0x42, // MOV R1, [...]
92         0x80,
93         0xAF, // CMP R10, R15
94     };
95     auto result = decompile(bytes, vram);
96
97     ASSERT_EQ(result.size(), 4);
98     EXPECT_EQ(result[0].offset(), 0u);
99     EXPECT_EQ(result[1].offset(), 2u);
100    EXPECT_EQ(result[2].offset(), 5u);
101    EXPECT_EQ(result[3].offset(), 11u);
102 }
103
104 TEST(decompile, DistinguishesNibbleVariantOpcodes) {
105     Vram vram = makeIdentityVram();
106     // 1B 0 ... -> MOV reg, addr
107     // 1B 1 ... -> MOV addr, reg (operand order swapped)
108     Bytes bytes{
109         0x1B,
110         0x01,
111         0x00,
112         0x00,
113         0x00,
114         0x42,
115         0x1B,
116         0x12,

```

```

117         0x00,
118         0x00,
119         0x00,
120         0x42,
121     };
122     auto result = decompile(bytes, vram);
123
124     ASSERT_EQ(result.size(), 2);
125     EXPECT_EQ(result[0].prototype.opcode, 0x1B0);
126     EXPECT_EQ(result[1].prototype.opcode, 0x1B1);
127 }
128
129 TEST(decompile, DistinguishesLit16AndLit32) {
130     Vram vram = makeIdentityVram();
131     // 1C 1 ... -> 4-byte form (lit16)
132     // 1C 2 ... -> 6-byte form (lit32)
133     Bytes bytes{
134         0x1C,
135         0x11,
136         0x33,
137         0x44,
138         0x1C,
139         0x21,
140         0x33,
141         0x44,
142         0x55,
143         0x66,
144     };
145     auto result = decompile(bytes, vram);
146
147     ASSERT_EQ(result.size(), 2);
148     EXPECT_EQ(result[0].prototype.opcode, 0x1C1);
149     EXPECT_EQ(result[0].size(), 4u);
150     EXPECT_EQ(result[1].prototype.opcode, 0x1C2);
151     EXPECT_EQ(result[1].size(), 6u);
152 }
153
154 TEST(decompile, ThrowsOnUnknownOpcode) {
155     Vram vram = makeIdentityVram();
156     Bytes bytes{0xFF, 0x00};
157
158     EXPECT_THROW(decompile(bytes, vram), DecompilerUnknownInstructionException);
159 }
160
161 TEST(decompile, ThrowsAfterDecodingValidPrefix) {
162     Vram vram = makeIdentityVram();
163     Bytes bytes{0x0A, 0x12, 0xFF, 0x00};
164
165     EXPECT_THROW(decompile(bytes, vram), DecompilerUnknownInstructionException);
166 }
167
168 TEST(decompile, PropagatesTranslationFailure) {
169     Vram vram = makeIdentityVram();
170     Bytes bytes{0x1B, 0x01, 0x00, 0x50, 0x00, 0x42};
171     auto result = decompile(bytes, vram);
172
173     ASSERT_EQ(result.size(), 1);
174     ASSERT_TRUE(result[0].error().has_value());
175     EXPECT_EQ(result[0].error()->error, TranslationError::SegmentNotFound);
176 }

```



```
1 #pragma once
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <optional>
6 #include <vector>
7
8 #include "assembly/InstructionPrototype/InstructionPrototype.hpp"
9 #include "vram/Vram/Vram.hpp"
10
11 class Instruction {
12 public:
13     const InstructionPrototype& prototype;
14
15     Instruction(
16         const InstructionPrototype& prototype,
17         std::vector<std::uint8_t> bytes,
18         std::size_t offset,
19         const Vram& vram
20     );
21
22     std::size_t offset() const;
23     std::size_t size() const;
24     std::optional<TranslationFailure> error() const;
25
26     std::vector<std::uint8_t> getOperandBytes(int operandIndex) const;
27
28 private:
29     std::vector<std::uint8_t> _bytes;
30     std::size_t _offset;
31     std::optional<TranslationFailure> _error;
32
33     void translate(const Vram& vram);
34     void setAddressBytes(int operandIndex, std::uint32_t value);
35 };
```

```
1 #include "assembly/Instruction/Instruction.hpp"
2
3 #include <utility>
4
5 Instruction::Instruction(
6     const InstructionPrototype& prototype,
7     std::vector<std::uint8_t> bytes,
8     std::size_t offset,
9     const Vram& vram
10 ) :
11     prototype(prototype), _bytes(std::move(bytes)), _offset(offset) {
12     translate(vram);
13 }
14
15 std::size_t Instruction::offset() const {
16     return _offset;
17 }
18
19 std::size_t Instruction::size() const {
20     return _bytes.size();
21 }
22
23 std::optional<TranslationFailure> Instruction::error() const {
24     return _error;
25 }
26
27 std::vector<std::uint8_t> Instruction::getOperandBytes(int operandIndex) const {
28     std::size_t offsetBits = prototype.opcodeBitWidth;
29
30     for (int i = 0; i < operandIndex; i++) {
31         offsetBits += getOperandBitWidth(prototype.operands[i]);
32     }
33
34     std::size_t operandBits =
35         getOperandBitWidth(prototype.operands[operandIndex]);
36     std::size_t operandBytes = (operandBits + 7) / 8;
37     std::vector<std::uint8_t> result(operandBytes);
38
39     for (std::size_t i = 0; i < operandBytes; i++) {
40         std::size_t bitIndex = offsetBits + i * 8;
41         std::size_t byteIndex = bitIndex / 8;
42
43         result[i] = _bytes[byteIndex];
44     }
45
46     bool isRegister = prototype.operands[operandIndex] == OperandType::Register;
47
48     if (isRegister) {
49         if (offsetBits % 8 == 0) {
50             result[0] >>= 4;
51         }
52
53         result[0] &= 0x0F;
54     }
55
56     return result;
```

```

57 }
58
59 void Instruction::translate(const Vram& vram) {
60     for (std::size_t i = 0; i < prototype.operands.size(); i++) {
61         if (prototype.operands[i] != OperandType::Address) {
62             continue;
63         }
64
65         std::vector<std::uint8_t> addressBytes = getOperandBytes(i);
66         std::uint32_t virtualAddress =
67             (std::uint32_t(addressBytes[0]) << 24) |
68             (std::uint32_t(addressBytes[1]) << 16) |
69             (std::uint32_t(addressBytes[2]) << 8) |
70             std::uint32_t(addressBytes[3]);
71
72         auto physical = vram.virtualToPhysical(virtualAddress);
73         if (!physical) {
74             _error = TranslationFailure{virtualAddress, physical.error()};
75             return;
76         }
77
78         setAddressBytes(i, *physical);
79         return;
80     }
81 }
82
83 void Instruction::setAddressBytes(int operandIndex, std::uint32_t value) {
84     std::size_t offsetBits = prototype.opcodeBitWidth;
85
86     for (int i = 0; i < operandIndex; i++) {
87         offsetBits += getOperandBitWidth(prototype.operands[i]);
88     }
89
90     std::size_t operandBytes = (getOperandBitWidth(OperandType::Address) + 7) / 8;
91
92     for (std::size_t i = 0; i < operandBytes; i++) {
93         std::size_t byteIndex = (offsetBits + i * 8) / 8;
94
95         _bytes[byteIndex] = (value >> (8 * (operandBytes - 1 - i))) & 0xFF;
96     }
97 }

```

```
1 #include "assembly/Instruction/Instruction.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <memory>
6 #include <vector>
7
8 #include "vram/PagesTable/PagesTable.hpp"
9 #include "vram/SegmentsTable/SegmentsTable.hpp"
10
11 using Bytes = std::vector<std::uint8_t>;
12
13 namespace {
14
15 Vram makeVram(std::uint32_t frame) {
16     std::vector<PageDescriptor> pages;
17     pages.push_back(PageDescriptor{.isResident = true, .frame = frame});
18
19     std::vector<SegmentDescriptor> segments;
20     segments.push_back(
21         SegmentDescriptor{
22             .pagesTable = std::make_unique<PagesTable>(std::move(pages))
23         }
24     );
25
26     return Vram(std::make_unique<SegmentsTable>(std::move(segments)));
27 }
28
29 } // namespace
30
31 TEST(Instruction, StoresPrototypeOffsetAndSize) {
32     Vram vram = makeVram(0);
33     InstructionPrototype proto(
34         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
35     );
36     Instruction instr(proto, {0x0A, 0x12}, 7, vram);
37
38     EXPECT_EQ(&instr.prototype, &proto);
39     EXPECT_EQ(instr.offset(), 7u);
40     EXPECT_EQ(instr.size(), 2u);
41 }
42
43 TEST(Instruction, GetOperandBytesIsolatesRegisterNibble) {
44     Vram vram = makeVram(0);
45     InstructionPrototype proto(
46         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
47     );
48     Instruction instr(proto, {0x0A, 0x12}, 0, vram);
49
50     EXPECT_FALSE(instr.error().has_value());
51     EXPECT_EQ(instr.getOperandBytes(0), Bytes{0x01});
52     EXPECT_EQ(instr.getOperandBytes(1), Bytes{0x02});
53 }
54
55 TEST(Instruction, TranslatesAddressOperandToPhysical) {
56     Vram vram = makeVram(0x10);
```

```

57     InstructionPrototype proto(
58         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
59     );
60     // seg 0 / page 0 / off 0x42, frame 0x10 -> (0x10 << 10) | 0x42 = 0x4042
61     Instruction instr(proto, {0x1B, 0x01, 0x00, 0x00, 0x00, 0x42}, 0, vram);
62
63     EXPECT_FALSE(instr.error().has_value());
64     EXPECT_EQ(instr.getOperandBytes(1), (Bytes{0x00, 0x00, 0x40, 0x42}));
65 }
66
67 TEST(Instruction, RecordsErrorAndKeepsVirtualAddressOnFailure) {
68     Vram vram = makeVram(0);
69     InstructionPrototype proto(
70         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
71     );
72     Instruction instr(proto, {0x1B, 0x01, 0x00, 0x50, 0x00, 0x42}, 0, vram);
73
74     ASSERT_TRUE(instr.error().has_value());
75     EXPECT_EQ(instr.error()->error, TranslationError::SegmentNotFound);
76     EXPECT_EQ(instr.error()->virtualAddress, 0x00500042u);
77     EXPECT_EQ(instr.getOperandBytes(1), (Bytes{0x00, 0x50, 0x00, 0x42}));
78 }

```

```
1 #pragma once
2
3 #include <string>
4
5 #include "assembly/Instruction/Instruction.hpp"
6
7 std::string formatInstruction(const Instruction& instruction);
```

```
1 #include "assembly/InstructionFormat/InstructionFormat.hpp"
2
3 #include "assembly/Operand/Operand.hpp"
4
5 std::string formatInstruction(const Instruction& instruction) {
6     const InstructionPrototype& prototype = instruction.prototype;
7
8     std::string result = prototype.format;
9
10    for (std::size_t i = 0; i < prototype.operands.size(); i++) {
11        std::string operandWildcard = '$' + std::to_string(i + 1);
12        std::size_t position = result.find(operandWildcard);
13
14        if (position == std::string::npos) {
15            break;
16        }
17
18        result.replace(
19            position,
20            operandWildcard.length(),
21            formatOperandBytes(
22                instruction.getOperandBytes(i), prototype.operands[i]
23            )
24        );
25    }
26
27    return result;
28 }
```

```
1 #pragma once
2
3 #include <stdint>
4 #include <string>
5 #include <vector>
6
7 #include "assembly/Operand/Operand.hpp"
8
9 class InstructionPrototype {
10 public:
11     const std::string format;
12     const std::uint16_t opcode;
13     const std::uint8_t opcodeBitWidth;
14     const std::vector<OperandType> operands;
15
16     std::size_t size() const;
17
18     InstructionPrototype(
19         std::string format,
20         std::uint16_t opcode,
21         std::uint8_t opcodeBitWidth,
22         std::vector<OperandType> operands = {}
23     );
24
25     bool doesMatch(const std::vector<std::uint8_t>& bytes) const;
26 };
27
28 const std::vector<InstructionPrototype> INSTRUCTION_PROTOTYPES = {
29     InstructionPrototype(
30         "MOV $1, $2",
31         0x1B0,
32         12,
33         {OperandType::Register, OperandType::Address}
34     ),
35     InstructionPrototype(
36         "MOV $2, $1",
37         0x1B1,
38         12,
39         {OperandType::Register, OperandType::Address}
40     ),
41     InstructionPrototype(
42         "ADD $1, $2, $3",
43         0x030,
44         12,
45         {OperandType::Register, OperandType::Register, OperandType::Register}
46     ),
47     InstructionPrototype(
48         "ADD $1, $2, $3",
49         0x04,
50         8,
51         {OperandType::Register, OperandType::Register, OperandType::Address}
52     ),
53     InstructionPrototype(
54         "SUB $1, $2",
55         0x0A,
56         8,
```



```

57         {OperandType::Register, OperandType::Register}
58     ),
59     InstructionPrototype(
60         "SUB $1, $2",
61         0x0B0,
62         12,
63         {OperandType::Register, OperandType::Address}
64     ),
65     InstructionPrototype(
66         "CMP $1, $2",
67         0x80,
68         8,
69         {OperandType::Register, OperandType::Register}
70     ),
71     InstructionPrototype(
72         "MOV $1, $2",
73         0x1C1,
74         12,
75         {OperandType::Register, OperandType::Literal16}
76     ),
77     InstructionPrototype(
78         "MOV $1, $2",
79         0x1C2,
80         12,
81         {OperandType::Register, OperandType::Literal32}
82     ),
83 };

```

```
1 #include "assembly/InstructionPrototype/InstructionPrototype.hpp"
2
3 std::size_t InstructionPrototype::size() const {
4     std::size_t sizeBits = opcodeBitWidth;
5
6     for (const auto& operand : operands) {
7         sizeBits += getOperandBitWidth(operand);
8     }
9
10    return (sizeBits + 7) / 8;
11 }
12
13 InstructionPrototype::InstructionPrototype(
14     std::string format,
15     std::uint16_t opcode,
16     std::uint8_t opcodeSize,
17     std::vector<OperandType> operands
18 ) :
19     format(format),
20     opcode(opcode),
21     opcodeBitWidth(opcodeSize),
22     operands(operands) {}
23
24 bool InstructionPrototype::doesMatch(
25     const std::vector<std::uint8_t>& bytes
26 ) const {
27     if (bytes.size() != size()) {
28         return false;
29     }
30
31     std::size_t opcodeBytes = (opcodeBitWidth + 7) / 8;
32     const std::size_t backshift = opcodeBytes * 8 - opcodeBitWidth;
33     std::uint16_t extractedOpcode = 0;
34
35     for (std::size_t i = 0; i < opcodeBytes; i++) {
36         extractedOpcode <<= 8;
37         extractedOpcode |= bytes[i];
38     }
39
40     extractedOpcode >>= backshift;
41
42     if (extractedOpcode != opcode) {
43         return false;
44     }
45
46     return true;
47 }
```

```
1 #include "assembly/InstructionPrototype/InstructionPrototype.hpp"
2
3 #include <gtest/gtest.h>
4
5 TEST(InstructionPrototype, ConstructorStoresFields) {
6     InstructionPrototype proto(
7         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
8     );
9
10    EXPECT_EQ(proto.format, "MOV $1, $2");
11    EXPECT_EQ(proto.opcode, 0x1B0);
12    EXPECT_EQ(proto.opcodeBitWidth, 12);
13    EXPECT_EQ(proto.operands.size(), 2);
14 }
15
16 TEST(InstructionPrototype, SizeEightBitOpcodeTwoRegs) {
17     // 8 + 4 + 4 = 16 bits = 2 bytes
18     InstructionPrototype proto(
19         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
20     );
21
22     EXPECT_EQ(proto.size(), 2);
23 }
24
25 TEST(InstructionPrototype, SizeTwelveBitOpcodeWithAddress) {
26     // 12 + 4 + 32 = 48 bits = 6 bytes
27     InstructionPrototype proto(
28         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
29     );
30
31     EXPECT_EQ(proto.size(), 6);
32 }
33
34 TEST(InstructionPrototype, SizeTwelveBitOpcodeThreeRegs) {
35     // 12 + 4 + 4 + 4 = 24 bits = 3 bytes
36     InstructionPrototype proto(
37         "ADD $1, $2, $3",
38         0x030,
39         12,
40         {OperandType::Register, OperandType::Register, OperandType::Register}
41     );
42
43     EXPECT_EQ(proto.size(), 3);
44 }
45
46 TEST(InstructionPrototype, SizeWithLit16) {
47     // 12 + 4 + 16 = 32 bits = 4 bytes
48     InstructionPrototype proto(
49         "MOV $1, $2", 0x1C1, 12, {OperandType::Register, OperandType::Literal16}
50     );
51
52     EXPECT_EQ(proto.size(), 4);
53 }
54
55 TEST(InstructionPrototype, SizeWithLit32) {
56     // 12 + 4 + 32 = 48 bits = 6 bytes
```

```

57     InstructionPrototype proto(
58         "MOV $1, $2", 0x1C2, 12, {OperandType::Register, OperandType::Literal32}
59     );
60
61     EXPECT_EQ(proto.size(), 6);
62 }
63
64 TEST(InstructionPrototype, DoesMatchEightBitOpcode) {
65     InstructionPrototype proto(
66         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
67     );
68
69     EXPECT_TRUE(proto.matches({0x0A, 0x12}));
70 }
71
72 TEST(InstructionPrototype, DoesMatchTwelveBitOpcode) {
73     InstructionPrototype proto(
74         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
75     );
76
77     EXPECT_TRUE(proto.matches({0x1B, 0x01, 0x00, 0x00, 0x00, 0x42}));
78 }
79
80 TEST(InstructionPrototype, DoesNotMatchWrongOpcode) {
81     InstructionPrototype proto(
82         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
83     );
84
85     EXPECT_FALSE(proto.matches({0x0B, 0x12}));
86 }
87
88 TEST(InstructionPrototype, DoesNotMatchWrongLength) {
89     InstructionPrototype proto(
90         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
91     );
92
93     EXPECT_FALSE(proto.matches({0x0A, 0x12, 0x00}));
94     EXPECT_FALSE(proto.matches({0x0A}));
95 }
96
97 TEST(InstructionPrototype, DistinguishesTwelveBitNibbles) {
98     InstructionPrototype proto1B0(
99         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
100     );
101     InstructionPrototype proto1B1(
102         "MOV $2, $1", 0x1B1, 12, {OperandType::Register, OperandType::Address}
103     );
104
105     std::vector<std::uint8_t> bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
106
107     EXPECT_TRUE(proto1B0.matches(bytes));
108     EXPECT_FALSE(proto1B1.matches(bytes));
109 }
110
111 TEST(InstructionPrototype, MatchIgnoresOperandNibble) {
112     InstructionPrototype proto(
113         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
114     );
115
116     EXPECT_TRUE(proto.matches({0x1B, 0x0F, 0x00, 0x00, 0x00, 0x42}));

```

```
117 }
118
119 TEST(InstructionPrototype, RegistryContainsNineVariantSixTemplates) {
120     EXPECT_EQ(INSTRUCTION_PROTOTYPES.size(), 9);
121 }
```

```
1 #pragma once
2
3 #include <stdint>
4 #include <stdexcept>
5 #include <string>
6 #include <vector>
7
8 enum class OperandType : std::uint8_t {
9     Register,
10    Address,
11    Literal16,
12    Literal32,
13 };
14
15 std::string formatOperandBytes(
16     const std::vector<std::uint8_t>& bytes,
17     OperandType type
18 );
19
20 std::string formatRegister(std::uint8_t byte);
21
22 std::string formatAddress(const std::vector<std::uint8_t>& bytes);
23
24 std::string formatLiteral16(const std::vector<std::uint8_t>& bytes);
25
26 std::string formatLiteral32(const std::vector<std::uint8_t>& bytes);
27
28 std::uint8_t getOperandBitWidth(OperandType type);
```

```
1 #include "assembly/Operand/Operand.hpp"
2
3 #include <bit>
4 #include <format>
5 #include <utility>
6
7 std::string formatOperandBytes(
8     const std::vector<std::uint8_t>& bytes,
9     OperandType type
10 ) {
11     switch (type) {
12         case OperandType::Register:
13             return formatRegister(bytes[0]);
14         case OperandType::Address:
15             return formatAddress(bytes);
16         case OperandType::Literal16:
17             return formatLiteral16(bytes);
18         case OperandType::Literal32:
19             return formatLiteral32(bytes);
20     }
21     std::unreachable();
22 }
23
24 std::string formatRegister(std::uint8_t byte) {
25     return std::format("R{}", byte & 0x0F);
26 }
27
28 std::string formatAddress(const std::vector<std::uint8_t>& bytes) {
29     std::uint32_t address =
30         (std::uint32_t(bytes[0]) << 24) | (std::uint32_t(bytes[1]) << 16) |
31         (std::uint32_t(bytes[2]) << 8) | std::uint32_t(bytes[3]);
32
33     return std::format("[0x{:08X}]", address);
34 }
35
36 std::string formatLiteral16(const std::vector<std::uint8_t>& bytes) {
37     std::uint16_t raw =
38         (std::uint16_t(bytes[0]) << 8) | std::uint16_t(bytes[1]);
39
40     return std::format("{} ", std::bit_cast<std::int16_t>(raw));
41 }
42
43 std::string formatLiteral32(const std::vector<std::uint8_t>& bytes) {
44     std::uint32_t raw =
45         (std::uint32_t(bytes[0]) << 24) | (std::uint32_t(bytes[1]) << 16) |
46         (std::uint32_t(bytes[2]) << 8) | std::uint32_t(bytes[3]);
47
48     return std::format("{} ", std::bit_cast<std::int32_t>(raw));
49 }
50
51 std::uint8_t getOperandBitWidth(OperandType type) {
52     switch (type) {
53         case OperandType::Register:
54             return 4;
55         case OperandType::Address:
```

```
57         return 32;
58     case OperandType::Literal16:
59         return 16;
60     case OperandType::Literal32:
61         return 32;
62 }
63
64 std::unreachable();
65 }
```



```
1 #pragma once
2
3 #include <stdexcept>
4 #include <string>
5 #include <unordered_map>
6 #include <vector>
7
8 class ArgsException : public std::runtime_error {
9 public:
10     using std::runtime_error::runtime_error;
11 };
12
13 class Args {
14 private:
15     struct StringArg {
16         std::string shortName;
17         bool required;
18         std::string value;
19         bool provided = false;
20     };
21
22     struct FlagArg {
23         std::string shortName;
24         bool value = false;
25     };
26
27     std::string _program;
28     std::vector<std::string> _argv;
29
30     std::unordered_map<std::string, StringArg> _strings;
31     std::unordered_map<std::string, FlagArg> _flags;
32     std::unordered_map<std::string, std::string> _aliases;
33
34 public:
35     Args(int argc, char* argv[]);
36
37     Args& expectString(
38         const std::string& name,
39         const std::string& shortName,
40         bool required = false
41     );
42     Args& expectFlag(const std::string& name, const std::string& shortName);
43
44     Args& parse();
45
46     void printUsage() const;
47
48     [[nodiscard]] const std::string& getString(const std::string& name) const;
49     [[nodiscard]] bool getFlag(const std::string& name) const;
50 };
```

```
1 #include "util/Args/Args.hpp"
2
3 #include <format>
4 #include <iostream>
5 #include <stdexcept>
6
7 Args::Args(int argc, char* argv[]) : _program(argv[0]) {
8     for (int i = 1; i < argc; i++) {
9         _argv.emplace_back(argv[i]);
10    }
11 }
12
13 Args& Args::expectString(
14     const std::string& name,
15     const std::string& shortName,
16     bool required
17 ) {
18     _strings[name] = {shortName, required, "", false};
19     _aliases["--" + name] = name;
20     _aliases["-" + shortName] = name;
21
22     return *this;
23 }
24
25 Args& Args::expectFlag(const std::string& name, const std::string& shortName) {
26     _flags[name] = {shortName, false};
27     _aliases["--" + name] = name;
28     _aliases["-" + shortName] = name;
29
30     return *this;
31 }
32
33 Args& Args::parse() {
34     for (size_t i = 0; i < _argv.size(); i++) {
35         const auto& arg = _argv[i];
36
37         auto alias = _aliases.find(arg);
38
39         if (alias == _aliases.end()) {
40             throw ArgsException("Unknown argument: " + arg);
41         }
42
43         const auto& name = alias->second;
44
45         auto stringArg = _strings.find(name);
46         if (stringArg != _strings.end()) {
47             if (i + 1 >= _argv.size()) {
48                 throw ArgsException("Missing value for: " + arg);
49             }
50
51             stringArg->second.value = _argv[++i];
52             stringArg->second.provided = true;
53
54             continue;
55         }
56     }
```

```

57     auto flagArg = _flags.find(name);
58     if (flagArg != _flags.end()) {
59         flagArg->second.value = true;
60     }
61 }
62
63 for (const auto& [name, arg] : _strings) {
64     if (arg.required && !arg.provided) {
65         throw ArgsException("Missing required argument: --" + name);
66     }
67 }
68
69 return *this;
70 }
71
72 void Args::printUsage() const {
73     std::cerr << "\nUsage: " << _program;
74
75     for (const auto& [name, arg] : _strings) {
76         if (arg.required) {
77             std::cerr << " --" << name << " <value>";
78         }
79     }
80
81     std::cerr << " [options]\n\nOptions:\n";
82
83     for (const auto& [name, arg] : _strings) {
84         std::cerr << std::format(
85             "  --{}, -{} <value>{}\n",
86             name,
87             arg.shortName,
88             arg.required ? " (required)" : ""
89         );
90     }
91
92     for (const auto& [name, arg] : _flags) {
93         std::cerr << std::format("  --{}, -{}\n", name, arg.shortName);
94     }
95 }
96
97 const std::string& Args::getString(const std::string& name) const {
98     auto it = _strings.find(name);
99
100     if (it == _strings.end()) {
101         throw std::runtime_error("Unknown string argument: " + name);
102     }
103
104     return it->second.value;
105 }
106
107 bool Args::getFlag(const std::string& name) const {
108     auto it = _flags.find(name);
109
110     if (it == _flags.end()) {
111         throw std::runtime_error("Unknown flag argument: " + name);
112     }
113
114     return it->second.value;
115 }

```

```
1 #include "util/Args/Args.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <string>
6 #include <vector>
7
8 namespace {
9
10 class MockArgv {
11 public:
12     explicit MockArgv(std::initializer_list<std::string> tokens) :
13         _tokens(tokens) {
14         for (auto& s : _tokens) {
15             _ptrs.push_back(s.data());
16         }
17     }
18
19     int argc() const {
20         return static_cast<int>(_ptrs.size());
21     }
22
23     char** argv() {
24         return _ptrs.data();
25     }
26
27 private:
28     std::vector<std::string> _tokens;
29     std::vector<char*> _ptrs;
30 };
31
32 } // namespace
33
34 TEST(Args, ParsesRequiredStringArg) {
35     MockArgv input{"/app", "--input", "foo.bin"};
36     Args args(input.argc(), input.argv());
37
38     args.expectString("input", "i", true).parse();
39
40     EXPECT_EQ(args.getString("input"), "foo.bin");
41 }
42
43 TEST(Args, ParsesShortFormStringArg) {
44     MockArgv input{"/app", "-i", "foo.bin"};
45     Args args(input.argc(), input.argv());
46
47     args.expectString("input", "i", true).parse();
48
49     EXPECT_EQ(args.getString("input"), "foo.bin");
50 }
51
52 TEST(Args, ParsesFlagLongForm) {
53     MockArgv input{"/app", "--hex"};
54     Args args(input.argc(), input.argv());
55
56     args.expectFlag("hex", "x").parse();
```

```

57
58     EXPECT_TRUE(args.getFlag("hex"));
59 }
60
61 TEST(Args, ParsesFlagShortForm) {
62     MockArgv input{"/app", "-x"};
63     Args args(input.argc(), input.argv());
64
65     args.expectFlag("hex", "x").parse();
66
67     EXPECT_TRUE(args.getFlag("hex"));
68 }
69
70 TEST(Args, UnsetFlagDefaultsToFalse) {
71     MockArgv input{"/app"};
72     Args args(input.argc(), input.argv());
73
74     args.expectFlag("hex", "x").parse();
75
76     EXPECT_FALSE(args.getFlag("hex"));
77 }
78
79 TEST(Args, OptionalStringDefaultsToEmpty) {
80     MockArgv input{"/app"};
81     Args args(input.argc(), input.argv());
82
83     args.expectString("output", "o").parse();
84
85     EXPECT_EQ(args.getString("output"), "");
86 }
87
88 TEST(Args, MixesStringsAndFlags) {
89     MockArgv input{"/app", "-i", "in.bin", "-x", "-o", "out.txt"};
90     Args args(input.argc(), input.argv());
91
92     args.expectString("input", "i", true)
93         .expectFlag("hex", "x")
94         .expectString("output", "o")
95         .parse();
96
97     EXPECT_EQ(args.getString("input"), "in.bin");
98     EXPECT_TRUE(args.getFlag("hex"));
99     EXPECT_EQ(args.getString("output"), "out.txt");
100 }
101
102 TEST(Args, OrderOfArgsDoesNotMatter) {
103     MockArgv input{"/app", "-o", "out.txt", "--hex", "--input", "in.bin"};
104     Args args(input.argc(), input.argv());
105
106     args.expectString("input", "i", true)
107         .expectFlag("hex", "x")
108         .expectString("output", "o")
109         .parse();
110
111     EXPECT_EQ(args.getString("input"), "in.bin");
112     EXPECT_TRUE(args.getFlag("hex"));
113     EXPECT_EQ(args.getString("output"), "out.txt");
114 }
115
116 TEST(Args, ThrowsOnMissingRequired) {

```

```

117     MockArgv input{"/app"};
118     Args args(input argc(), input argv());
119
120     args.expectString("input", "i", true);
121
122     EXPECT_THROW(args.parse(), ArgsException);
123 }
124
125 TEST(Args, ThrowsOnUnknownArg) {
126     MockArgv input{"/app", "--bogus"};
127     Args args(input argc(), input argv());
128
129     EXPECT_THROW(args.parse(), ArgsException);
130 }
131
132 TEST(Args, ThrowsOnMissingValueForStringArg) {
133     MockArgv input{"/app", "--input"};
134     Args args(input argc(), input argv());
135
136     args.expectString("input", "i", true);
137
138     EXPECT_THROW(args.parse(), ArgsException);
139 }
140
141 TEST(Args, GetStringThrowsOnUnknownName) {
142     MockArgv input{"/app"};
143     Args args(input argc(), input argv());
144
145     args.parse();
146
147     EXPECT_THROW(args.getString("never-registered"), std::runtime_error);
148 }
149
150 TEST(Args, GetFlagThrowsOnUnknownName) {
151     MockArgv input{"/app"};
152     Args args(input argc(), input argv());
153
154     args.parse();
155
156     EXPECT_THROW(args.getFlag("never-registered"), std::runtime_error);
157 }
158
159 TEST(Args, LaterValueOverridesEarlier) {
160     MockArgv input{"/app", "-i", "first.bin", "-i", "second.bin"};
161     Args args(input argc(), input argv());
162
163     args.expectString("input", "i", true).parse();
164
165     EXPECT_EQ(args.getString("input"), "second.bin");
166 }

```

```
1 #pragma once
2
3 #include <fstream>
4 #include <string>
5 #include <vector>
6
7 class Csv {
8 private:
9     std::vector<std::vector<std::string>> _rows;
10
11 public:
12     Csv(std::ifstream stream);
13
14     const std::vector<std::vector<std::string>>& rows();
15 };
```

```
1 #include "util/Csv/Csv.hpp"
2
3 #include "util/String/String.hpp"
4
5 Csv::Csv(std::ifstream stream) {
6     std::string buffer = "";
7
8     while (std::getline(stream, buffer)) {
9         _rows.push_back(String::split(buffer, ','));
10    }
11 }
12
13 const std::vector<std::vector<std::string>>& Csv::rows() {
14     return _rows;
15 }
```



```
1 #pragma once
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 std::vector<std::uint8_t> hexToBytes(std::string_view hex);
10
11 std::string bytesToHex(const std::vector<std::uint8_t>& bytes);
12
13 std::string bytesToHex(
14     const std::vector<std::uint8_t>& bytes,
15     std::size_t offset,
16     std::size_t count
17 );
```

```
1 #include "util/Hex/Hex.hpp"
2
3 #include <cstdint>
4 #include <format>
5 #include <stdexcept>
6
7 namespace {
8
9 int hexDigitValue(char nibble) {
10     if (nibble >= '0' && nibble <= '9') {
11         return nibble - '0';
12     }
13
14     if (nibble >= 'A' && nibble <= 'F') {
15         return nibble - 'A' + 10;
16     }
17
18     if (nibble >= 'a' && nibble <= 'f') {
19         return nibble - 'a' + 10;
20     }
21
22     return -1;
23 }
24
25 } // namespace
26
27 std::vector<std::uint8_t> hexToBytes(std::string_view hex) {
28     std::vector<std::uint8_t> result;
29     int high = -1;
30
31     for (std::size_t i = 0; i < hex.size(); i++) {
32         if (hex[i] == '/' && i + 1 < hex.size() && hex[i + 1] == '/') {
33             while (i < hex.size() && hex[i] != '\n') {
34                 i++;
35             }
36
37             continue;
38         }
39
40         int value = hexDigitValue(hex[i]);
41
42         if (value < 0) {
43             continue;
44         }
45
46         if (high < 0) {
47             high = value;
48
49             continue;
50         }
51
52         result.push_back(static_cast<std::uint8_t>((high << 4) | value));
53         high = -1;
54     }
55
56     if (high >= 0) {
```

```

57         throw std::invalid_argument("hexToBytes: odd number of hex digits");
58     }
59
60     return result;
61 }
62
63 std::string bytesToHex(
64     const std::vector<std::uint8_t>& bytes,
65     std::size_t offset,
66     std::size_t count
67 ) {
68     std::string result;
69
70     for (std::size_t i = 0; i < count; i++) {
71         result += std::format("{:02X} ", bytes[offset + i]);
72     }
73
74     return result;
75 }
76
77 std::string bytesToHex(const std::vector<std::uint8_t>& bytes) {
78     return bytesToHex(bytes, 0, bytes.size());
79 }

```

```
1 #include "util/Hex/Hex.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <stdexcept>
6
7 using Bytes = std::vector<std::uint8_t>;
8
9 TEST(hexToBytes, Empty) {
10     EXPECT_EQ(hexToBytes(""), Bytes{});
11 }
12
13 TEST(hexToBytes, WhitespaceOnly) {
14     EXPECT_EQ(hexToBytes("  \t\n  "), Bytes{});
15 }
16
17 TEST(hexToBytes, SingleByte) {
18     EXPECT_EQ(hexToBytes("0A"), Bytes{0x0A});
19     EXPECT_EQ(hexToBytes("FF"), Bytes{0xFF});
20 }
21
22 TEST(hexToBytes, CaseInsensitive) {
23     EXPECT_EQ(hexToBytes("ab"), Bytes{0xAB});
24     EXPECT_EQ(hexToBytes("Ab"), Bytes{0xAB});
25     EXPECT_EQ(hexToBytes("AB"), Bytes{0xAB});
26 }
27
28 TEST(hexToBytes, SpaceSeparated) {
29     EXPECT_EQ(hexToBytes("0A 12 80 AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
30 }
31
32 TEST(hexToBytes, NoSeparators) {
33     EXPECT_EQ(hexToBytes("0A1280AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
34 }
35
36 TEST(hexToBytes, MixedWhitespace) {
37     EXPECT_EQ(hexToBytes("0A\t12\n80\r\nAF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
38 }
39
40 TEST(hexToBytes, OddDigitCountThrows) {
41     EXPECT_THROW(hexToBytes("A"), std::invalid_argument);
42     EXPECT_THROW(hexToBytes("0A 12 8"), std::invalid_argument);
43 }
44
45 TEST(hexToBytes, NonHexCharsAreSkipped) {
46     EXPECT_EQ(hexToBytes("0A, 12; 80 | AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
47 }
48
49 TEST(hexToBytes, LineCommentIsSkipped) {
50     EXPECT_EQ(
51         hexToBytes("0A 12 // SUB R1, R2\n80 AF // CMP R10, R15\n"),
52         (Bytes{0x0A, 0x12, 0x80, 0xAF})
53     );
54 }
55
56 TEST(hexToBytes, FullLineComment) {
```

```

57     EXPECT_EQ(
58         hexToBytes("// this is a header comment\n0A 12\n"), (Bytes{0x0A, 0x12})
59     );
60 }
61
62 TEST(hexToBytes, CommentSkipsHexLikeChars) {
63     EXPECT_EQ(
64         hexToBytes("0A 12 // AF BE EF\n80 AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF})
65     );
66 }
67
68 TEST(hexToBytes, CommentAtEndOfInputWithoutNewline) {
69     EXPECT_EQ(hexToBytes("0A 12 // no trailing newline"), (Bytes{0x0A, 0x12}));
70 }
71
72 TEST(hexToBytes, SingleSlashIsNotAComment) {
73     EXPECT_EQ(hexToBytes("0A / 12"), (Bytes{0x0A, 0x12}));
74 }
75
76 TEST(bytesToHex, Empty) {
77     EXPECT_EQ(bytesToHex({}), "");
78 }
79
80 TEST(bytesToHex, SingleByte) {
81     EXPECT_EQ(bytesToHex({0x0A}), "0A ");
82     EXPECT_EQ(bytesToHex({0xFF}), "FF ");
83 }
84
85 TEST(bytesToHex, MultipleBytes) {
86     EXPECT_EQ(bytesToHex({0x0A, 0x12, 0x80, 0xAF}), "0A 12 80 AF ");
87 }
88
89 TEST(bytesToHex, ZeroPadding) {
90     EXPECT_EQ(bytesToHex({0x00, 0x01, 0x0F}), "00 01 0F ");
91 }
92
93 TEST(bytesToHex, UppercaseHex) {
94     EXPECT_EQ(bytesToHex({0xAB, 0xCD, 0xEF}), "AB CD EF ");
95 }
96
97 TEST(Roundtrip, BytesToHexToBytes) {
98     Bytes original{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
99     EXPECT_EQ(hexToBytes(bytesToHex(original)), original);
100 }
101
102 TEST(Roundtrip, HexToBytesToHex) {
103     std::string normalized = "1B 01 00 00 00 42 ";
104     EXPECT_EQ(bytesToHex(hexToBytes(normalized)), normalized);
105 }

```

```
1 #pragma once
2
3 #include <string>
4 #include <vector>
5
6 class String {
7 public:
8     String() = delete;
9
10     static std::vector<std::string> split(std::string str, char delimiter);
11 };
```

```
1 #include "util/String/String.hpp"
2
3 #include <utility>
4
5 std::vector<std::string> String::split(std::string str, char delimiter) {
6     std::vector<std::string> parts;
7     std::size_t start = 0;
8
9     while (true) {
10         std::size_t pos = str.find(delimiter, start);
11
12         if (pos == std::string::npos) {
13             parts.push_back(str.substr(start));
14
15             break;
16         }
17
18         parts.push_back(str.substr(start, pos - start));
19         start = pos + 1;
20     }
21
22     return parts;
23 }
```

```
1 #include "util/String/String.hpp"
2
3 #include <gtest/gtest.h>
4
5 using Parts = std::vector<std::string>;
6
7 TEST(split, NoDelimiter) {
8     EXPECT_EQ(String::split("abc", ','), (Parts{"abc"}));
9 }
10
11 TEST(split, TwoParts) {
12     EXPECT_EQ(String::split("a,b", ','), (Parts{"a", "b"}));
13 }
14
15 TEST(split, MultipleParts) {
16     EXPECT_EQ(String::split("a,b,c,d", ','), (Parts{"a", "b", "c", "d"}));
17 }
18
19 TEST(split, LeadingDelimiter) {
20     EXPECT_EQ(String::split(",a", ','), (Parts{"", "a"}));
21 }
22
23 TEST(split, TrailingDelimiter) {
24     EXPECT_EQ(String::split("a,", ','), (Parts{"a", ""}));
25 }
26
27 TEST(split, ConsecutiveDelimiters) {
28     EXPECT_EQ(String::split("a,,b", ','), (Parts{"a", "", "b"}));
29 }
30
31 TEST(split, OnlyDelimiters) {
32     EXPECT_EQ(String::split(",,, ", ','), (Parts{"", "", ""}));
33 }
34
35 TEST(split, SingleDelimiter) {
36     EXPECT_EQ(String::split(",,", ','), (Parts{"", ""}));
37 }
38
39 TEST(split, NewlineDelimiter) {
40     EXPECT_EQ(
41         String::split("line1\nline2\n", '\n'), (Parts{"line1", "line2", ""})
42     );
43 }
44
45 TEST(split, PreservesWhitespace) {
46     EXPECT_EQ(String::split(" a , b ", ','), (Parts{" a ", " b "}));
47 }
48
49 TEST(split, PartCountIsDelimiterCountPlusOne) {
50     EXPECT_EQ(String::split("a,b,c", ',').size(), 3u);
51     EXPECT_EQ(String::split("a;b;c;d;e", ';').size(), 5u);
52 }
```



```
1 #pragma once
2
3 #include <cstdint>
4 #include <vector>
5
6 struct PageDescriptor {
7     bool isResident;
8     std::uint32_t frame;
9 };
10
11 class PagesTable {
12 private:
13     std::vector<PageDescriptor> _entries;
14
15 public:
16     PagesTable(std::vector<PageDescriptor> entries);
17
18     const PageDescriptor* get(std::uint16_t id) const;
19 };
```

```
1 #include "vram/PagesTable/PagesTable.hpp"
2
3 #include <utility>
4
5 PagesTable::PagesTable(std::vector<PageDescriptor> entries) :
6     _entries(std::move(entries)) {}
7
8 const PageDescriptor* PagesTable::get(std::uint16_t id) const {
9     if (id >= _entries.size()) {
10         return nullptr;
11     }
12
13     return &_amp;_entries[id];
14 }
```

```
1 #include "vram/PagesTable/PagesTable.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <vector>
6
7 namespace {
8
9   PagesTable makePagesTable() {
10     std::vector<PageDescriptor> pages;
11     pages.push_back(PageDescriptor{.isResident = true, .frame = 16});
12     pages.push_back(PageDescriptor{.isResident = false, .frame = 0});
13     pages.push_back(PageDescriptor{.isResident = true, .frame = 18});
14
15     return PagesTable(std::move(pages));
16   }
17
18 } // namespace
19
20 TEST(PagesTable, GetReturnsDescriptorForPageInRange) {
21   PagesTable table = makePagesTable();
22
23   const PageDescriptor* page = table.get(0);
24
25   ASSERT_NE(page, nullptr);
26   EXPECT_TRUE(page->isResident);
27   EXPECT_EQ(page->frame, 16u);
28 }
29
30 TEST(PagesTable, GetPreservesNonResidentFlag) {
31   PagesTable table = makePagesTable();
32
33   const PageDescriptor* page = table.get(1);
34
35   ASSERT_NE(page, nullptr);
36   EXPECT_FALSE(page->isResident);
37 }
38
39 TEST(PagesTable, GetReturnsNullptrForPageBeyondSegment) {
40   PagesTable table = makePagesTable();
41
42   EXPECT_EQ(table.get(3), nullptr);
43 }
```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <memory>
5 #include <vector>
6
7 #include "vram/PagesTable/PagesTable.hpp"
8
9 struct SegmentDescriptor {
10     std::unique_ptr<PagesTable> pagesTable;
11 };
12
13 class SegmentsTable {
14 private:
15     std::vector<SegmentDescriptor> _entries;
16
17 public:
18     SegmentsTable(std::vector<SegmentDescriptor> entries);
19
20     const SegmentDescriptor* get(std::uint16_t id) const;
21 };
```

```
1 #include "vram/SegmentsTable/SegmentsTable.hpp"
2
3 #include <utility>
4
5 SegmentsTable::SegmentsTable(std::vector<SegmentDescriptor> entries) :
6     _entries(std::move(entries)) {}
7
8 const SegmentDescriptor* SegmentsTable::get(std::uint16_t id) const {
9     if (id >= _entries.size()) {
10         return nullptr;
11     }
12
13     return &_amp;_entries[id];
14 }
```

```
1 #include "vram/SegmentsTable/SegmentsTable.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <memory>
6 #include <vector>
7
8 namespace {
9
10 SegmentsTable makeSegmentsTable() {
11     std::vector<PageDescriptor> pages;
12     pages.push_back(PageDescriptor{.isResident = true, .frame = 1});
13
14     std::vector<SegmentDescriptor> segments;
15     segments.push_back(
16         SegmentDescriptor{
17             .pagesTable = std::make_unique<PagesTable>(std::move(pages))
18         }
19     );
20
21     return SegmentsTable(std::move(segments));
22 }
23
24 } // namespace
25
26 TEST(SegmentsTable, GetReturnsDescriptorForSegmentInRange) {
27     SegmentsTable table = makeSegmentsTable();
28
29     const SegmentDescriptor* segment = table.get(0);
30
31     ASSERT_NE(segment, nullptr);
32     ASSERT_NE(segment->pagesTable, nullptr);
33     EXPECT_NE(segment->pagesTable->get(0), nullptr);
34 }
35
36 TEST(SegmentsTable, GetReturnsNullptrForSegmentBeyondTable) {
37     SegmentsTable table = makeSegmentsTable();
38
39     EXPECT_EQ(table.get(1), nullptr);
40 }
```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <expected>
5 #include <memory>
6 #include <string>
7
8 #include "vram/SegmentsTable/SegmentsTable.hpp"
9
10 enum class TranslationError {
11     SegmentNotFound,
12     PageNotFound,
13     PageNotResident,
14 };
15
16 struct TranslationFailure {
17     std::uint32_t virtualAddress;
18     TranslationError error;
19 };
20
21 std::string describeTranslationError(
22     std::uint32_t virtualAddress,
23     TranslationError error
24 );
25
26 class Vram {
27 private:
28     std::unique_ptr<SegmentsTable> _segmentsTable;
29
30 public:
31     Vram(std::unique_ptr<SegmentsTable> segmentsTable);
32
33     std::expected<std::uint32_t, TranslationError> virtualToPhysical(
34         std::uint32_t vaddr
35     ) const;
36 };
```

```
1 #include "vram/Vram/Vram.hpp"
2
3 #include <format>
4 #include <string>
5 #include <utility>
6
7 namespace {
8
9     const std::uint32_t SEGMENT_ID_MASK = 0xFFF00000;
10    const std::uint8_t SEGMENT_ID_SIZE = 12;
11
12    const std::uint32_t PAGE_ID_MASK = 0x000FFC00;
13    const std::uint8_t PAGE_ID_SIZE = 10;
14
15    const std::uint32_t OFFSET_MASK = 0x000003FF;
16    const std::uint8_t OFFSET_SIZE = 10;
17
18 } // namespace
19
20 Vram::Vram(std::unique_ptr<SegmentsTable> segmentsTable) :
21     _segmentsTable(std::move(segmentsTable)) {}
22
23 std::expected<std::uint32_t, TranslationError> Vram::virtualToPhysical(
24     std::uint32_t vaddr
25 ) const {
26     std::uint16_t segmentId =
27         (vaddr & SEGMENT_ID_MASK) >> (PAGE_ID_SIZE + OFFSET_SIZE);
28     std::uint16_t pageId = (vaddr & PAGE_ID_MASK) >> OFFSET_SIZE;
29     std::uint16_t offset = (vaddr & OFFSET_MASK);
30
31     auto segment = _segmentsTable->get(segmentId);
32     if (!segment) {
33         return std::unexpected(TranslationError::SegmentNotFound);
34     }
35
36     auto page = segment->pagesTable->get(pageId);
37     if (!page) {
38         return std::unexpected(TranslationError::PageNotFound);
39     }
40
41     if (!page->isResident) {
42         return std::unexpected(TranslationError::PageNotResident);
43     }
44
45     return (page->frame << OFFSET_SIZE) | offset;
46 }
47
48 std::string describeTranslationError(
49     std::uint32_t virtualAddress,
50     TranslationError error
51 ) {
52     std::uint16_t segmentId =
53         (virtualAddress & SEGMENT_ID_MASK) >> (PAGE_ID_SIZE + OFFSET_SIZE);
54     std::uint16_t pageId = (virtualAddress & PAGE_ID_MASK) >> OFFSET_SIZE;
55
56     std::string reason;
```



```
57     switch (error) {
58         case TranslationError::SegmentNotFound:
59             reason = std::format("segment [{}] doesn't exist", segmentId);
60             break;
61         case TranslationError::PageNotFound:
62             reason = std::format("page [{}] doesn't exist", pageId);
63             break;
64         case TranslationError::PageNotResident:
65             reason = std::format("page [{}] is not resident", pageId);
66             break;
67     }
68
69     return std::format(
70         "failed to convert virtual address [0x{:08X}] because {}",
71         virtualAddress,
72         reason
73     );
74 }
```

```
1 #include "vram/Vram/Vram.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <cstdint>
6 #include <memory>
7 #include <vector>
8
9 #include "vram/PagesTable/PagesTable.hpp"
10 #include "vram/SegmentsTable/SegmentsTable.hpp"
11
12 namespace {
13
14 std::uint32_t makeVaddr(
15     std::uint16_t segment,
16     std::uint16_t page,
17     std::uint16_t offset
18 ) {
19     return (std::uint32_t(segment) << 20) | (std::uint32_t(page) << 10) |
20         offset;
21 }
22
23 Vram makeVram() {
24     std::vector<PageDescriptor> seg0Pages;
25     seg0Pages.push_back(PageDescriptor{.isResident = true, .frame = 0x5});
26     seg0Pages.push_back(PageDescriptor{.isResident = false, .frame = 0});
27     seg0Pages.push_back(PageDescriptor{.isResident = true, .frame = 0x7});
28
29     std::vector<PageDescriptor> seg1Pages;
30     seg1Pages.push_back(PageDescriptor{.isResident = true, .frame = 0x9});
31
32     std::vector<SegmentDescriptor> segments;
33     segments.push_back(
34         SegmentDescriptor{
35             .pagesTable = std::make_unique<PagesTable>(std::move(seg0Pages))
36         }
37     );
38     segments.push_back(
39         SegmentDescriptor{
40             .pagesTable = std::make_unique<PagesTable>(std::move(seg1Pages))
41         }
42     );
43
44     return Vram(std::make_unique<SegmentsTable>(std::move(segments)));
45 }
46
47 } // namespace
48
49 TEST(Vram, TranslatesResidentPage) {
50     Vram vram = makeVram();
51
52     auto result = vram.virtualToPhysical(makeVaddr(0, 0, 0xAB));
53
54     ASSERT_TRUE(result.has_value());
55     EXPECT_EQ(*result, (std::uint32_t(0x5) << 10) | 0xAB);
56 }
```

```
57
58 TEST(Vram, PreservesOffsetWithinPage) {
59     Vram vram = makeVram();
60
61     auto result = vram.virtualToPhysical(makeVaddr(0, 2, 0x3FF));
62
63     ASSERT_TRUE(result.has_value());
64     EXPECT_EQ(*result & 0x3FF, 0x3FFu);
65 }
66
67 TEST(Vram, UnknownSegmentReturnsError) {
68     Vram vram = makeVram();
69
70     auto result = vram.virtualToPhysical(makeVaddr(2, 0, 0));
71
72     ASSERT_FALSE(result.has_value());
73     EXPECT_EQ(result.error(), TranslationError::SegmentNotFound);
74 }
75
76 TEST(Vram, PageBeyondSegmentReturnsError) {
77     Vram vram = makeVram();
78
79     auto result = vram.virtualToPhysical(makeVaddr(0, 3, 0));
80
81     ASSERT_FALSE(result.has_value());
82     EXPECT_EQ(result.error(), TranslationError::PageNotFound);
83 }
84
85 TEST(Vram, NonResidentPageReturnsError) {
86     Vram vram = makeVram();
87
88     auto result = vram.virtualToPhysical(makeVaddr(0, 1, 0));
89
90     ASSERT_FALSE(result.has_value());
91     EXPECT_EQ(result.error(), TranslationError::PageNotResident);
92 }
```

src\vram\VramImage\VramImage.hpp

```
1 #pragma once
2
3 #include <filesystem>
4
5 #include "vram/Vram/Vram.hpp"
6
7 class VramImage {
8 public:
9     static Vram load(const std::filesystem::path& path);
10 };
```

```
1 #include "vram/VramImage/VramImage.hpp"
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <format>
6 #include <fstream>
7 #include <memory>
8 #include <stdexcept>
9 #include <string>
10 #include <utility>
11 #include <vector>
12
13 #include "util/Csv/Csv.hpp"
14 #include "vram/PagesTable/PagesTable.hpp"
15 #include "vram/SegmentsTable/SegmentsTable.hpp"
16
17 namespace {
18
19     const std::size_t MAX_SEGMENTS = 4096;
20     const std::size_t MAX_PAGES = 1024;
21
22     PagesTable loadPagesTable(const std::filesystem::path& path) {
23         std::ifstream file(path);
24         Csv csv(std::move(file));
25
26         std::vector<PageDescriptor> pages;
27
28         for (const auto& row : csv.rows()) {
29             bool isResident = std::stoul(row.at(1)) > 0;
30             std::uint32_t frame = std::stoul(row.at(2));
31
32             pages.push_back(
33                 PageDescriptor{.isResident = isResident, .frame = frame}
34             );
35         }
36
37         if (pages.size() > MAX_PAGES) {
38             throw std::runtime_error(
39                 std::format(
40                     "pages table '{}' has {} entries, exceeding the {}-page limit",
41                     path.string(),
42                     pages.size(),
43                     MAX_PAGES
44                 )
45             );
46         }
47
48         return PagesTable(std::move(pages));
49     }
50
51 } // namespace
52
53 Vram VramImage::load(const std::filesystem::path& path) {
54     std::ifstream file(path);
55     Csv csv(std::move(file));
56 }
```

```

57     std::vector<SegmentDescriptor> segments;
58
59     for (const auto& row : csv.rows()) {
60         segments.push_back(
61             SegmentDescriptor{
62                 .pagesTable =
63                     std::make_unique<PagesTable>(loadPagesTable(row.at(0)))
64             }
65         );
66     }
67
68     if (segments.size() > MAX_SEGMENTS) {
69         throw std::runtime_error(
70             std::format(
71                 "segments table '{}' has {} entries, exceeding the {}-segment "
72                 "limit",
73                 path.string(),
74                 segments.size(),
75                 MAX_SEGMENTS
76             )
77         );
78     }
79
80     return Vram(std::make_unique<SegmentsTable>(std::move(segments)));
81 }

```

```
1 #include <gtest/gtest.h>
2
3 #include <memory>
4 #include <string>
5 #include <string_view>
6 #include <vector>
7
8 #include "assembly/Decompiler/Decompiler.hpp"
9 #include "assembly/InstructionFormat/InstructionFormat.hpp"
10 #include "util/Hex/Hex.hpp"
11 #include "vram/PagesTable/PagesTable.hpp"
12 #include "vram/SegmentsTable/SegmentsTable.hpp"
13 #include "vram/Vram/Vram.hpp"
14
15 namespace {
16
17 Vram makeIdentityVram() {
18     std::vector<PageDescriptor> pages;
19     pages.push_back(PageDescriptor{.isResident = true, .frame = 0});
20
21     std::vector<SegmentDescriptor> segments;
22     segments.push_back(
23         SegmentDescriptor{
24             .pagesTable = std::make_unique<PagesTable>(std::move(pages))
25         }
26     );
27
28     return Vram(std::make_unique<SegmentsTable>(std::move(segments)));
29 }
30
31 std::string decodeSingle(std::string_view hex) {
32     Vram vram = makeIdentityVram();
33     auto bytes = hexToBytes(hex);
34     auto instructions = decompile(bytes, vram);
35
36     return formatInstruction(instructions[0]);
37 }
38
39 std::vector<std::string> decodeAll(std::string_view hex) {
40     Vram vram = makeIdentityVram();
41     auto bytes = hexToBytes(hex);
42     auto instructions = decompile(bytes, vram);
43
44     std::vector<std::string> result;
45     for (const auto& instr : instructions) {
46         result.push_back(formatInstruction(instr));
47     }
48
49     return result;
50 }
51
52 } // namespace
53
54 TEST(Integration, Template2_MovRegFromAddr) {
55     EXPECT_EQ(decodeSingle("1B 01 00 00 00 42"), "MOV R1, [0x00000042]");
56 }
```

```

57
58 TEST(Integration, Template3_MovAddrFromReg) {
59     EXPECT_EQ(decodeSingle("1B 12 00 00 00 42"), "MOV [0x00000042], R2");
60 }
61
62 TEST(Integration, Template6_AddRegRegReg) {
63     EXPECT_EQ(decodeSingle("03 01 23"), "ADD R1, R2, R3");
64 }
65
66 TEST(Integration, Template7_AddRegRegAddr) {
67     EXPECT_EQ(decodeSingle("04 12 00 00 00 42"), "ADD R1, R2, [0x00000042]");
68 }
69
70 TEST(Integration, Template8_SubRegReg) {
71     EXPECT_EQ(decodeSingle("0A 12"), "SUB R1, R2");
72 }
73
74 TEST(Integration, Template9_SubRegAddr) {
75     EXPECT_EQ(decodeSingle("0B 01 00 00 00 42"), "SUB R1, [0x00000042]");
76 }
77
78 TEST(Integration, Template26_CmpRegReg) {
79     EXPECT_EQ(decodeSingle("80 AF"), "CMP R10, R15");
80 }
81
82 TEST(Integration, Template28_MovRegLit16Positive) {
83     EXPECT_EQ(decodeSingle("1C 11 33 44"), "MOV R1, 13124");
84 }
85
86 TEST(Integration, Template28_MovRegLit16Negative) {
87     EXPECT_EQ(decodeSingle("1C 1F FF FF"), "MOV R15, -1");
88 }
89
90 TEST(Integration, Template29_MovRegLit32Positive) {
91     EXPECT_EQ(decodeSingle("1C 21 33 44 55 66"), "MOV R1, 860116326");
92 }
93
94 TEST(Integration, Template29_MovRegLit32Negative) {
95     EXPECT_EQ(decodeSingle("1C 2F FF FF FF FF"), "MOV R15, -1");
96 }
97
98 TEST(Integration, AllTemplatesInSingleStream) {
99     auto asm_lines = decodeAll(
100         "1B 01 00 00 00 42 " // template 2 : MOV R1, [0x00000042]
101         "1B 12 00 00 00 42 " // template 3 : MOV [0x00000042], R2
102         "03 01 23 " // template 6 : ADD R1, R2, R3
103         "04 12 00 00 00 42 " // template 7 : ADD R1, R2, [0x00000042]
104         "0A 12 " // template 8 : SUB R1, R2
105         "0B 01 00 00 00 42 " // template 9 : SUB R1, [0x00000042]
106         "80 AF " // template 26 : CMP R10, R15
107         "1C 11 33 44 " // template 28 : MOV R1, 13124
108         "1C 21 33 44 55 66" // template 29 : MOV R1, 860116326
109     );
110
111     EXPECT_EQ(
112         asm_lines,
113         (std::vector<std::string>{
114             "MOV R1, [0x00000042]",
115             "MOV [0x00000042], R2",
116             "ADD R1, R2, R3",

```



```
117         "ADD R1, R2, [0x00000042]",
118         "SUB R1, R2",
119         "SUB R1, [0x00000042]",
120         "CMP R10, R15",
121         "MOV R1, 13124",
122         "MOV R1, 860116326",
123     })
124 );
125 }
```